PRÁCTICA GUIADA

MeteoLocal 0.3: entendemos cómo se organiza una app multipantalla

Modalidad	Individual	**Duración**	90–110 minutos
Unidad	U2. Desarrollo de aplicaciones y persistencia	**Producto**	MeteoLocal 0.3 funcionando y comprendida

OBJETIVO. Transformar MeteoLocal 0.2 en una aplicación con tres pantallas y comprender cómo se coordinan el estado, la navegación, los menús y los diálogos en Jetpack Compose.

IMPORTANTE. Partimos de que esta es tu primera experiencia programando aplicaciones Android. No se espera que conozcas de antemano términos como NavController, NavHost o Scaffold: los iremos introduciendo justo antes de utilizarlos.

MÉTODO DE TRABAJO. No se ofrece al principio una solución completa para copiar. Construiremos la aplicación por bloques. Cada bloque explicará qué problema resuelve, el concepto nuevo que aparece, dónde se inserta el código y qué debes observar después.

1. Qué problema vamos a resolver

MeteoLocal 0.2 tiene una sola pantalla. En 0.3 queremos que empiece a comportarse como una aplicación real con varias secciones.

```
MeteoLocal 0.2
└─ una sola pantalla

MeteoLocal 0.3
├─ Consulta
├─ Favoritos
└─ Configuración
```

Además, añadiremos dos tipos de interacción que no cambian de pantalla: un diálogo de confirmación para Limpiar y un menú «Más» con información sobre la aplicación.

CONCEPTO NUEVO · Pantalla @Composable

En una aplicación construida con Jetpack Compose, una pantalla puede ser simplemente una función marcada con @Composable. No necesitamos crear una Activity diferente para Consulta, Favoritos y Configuración.

```
ConsultaScreen()
FavoritosScreen()
ConfiguracionScreen()
      ↓
tres funciones @Composable
```

2. Antes de modificar nada

1	**Ejecuta MeteoLocal 0.2.** Comprueba que puedes escribir una localidad, pulsar Consultar, ver el mensaje y utilizar Limpiar. Si la versión anterior falla, corrígela antes de seguir.

2	**Conserva una copia estable.** Vamos a reorganizar bastante el proyecto. Mantén un punto de retorno de MeteoLocal 0.2 siguiendo el procedimiento indicado por el profesor.

3. Nuestro mapa de archivos

Hasta ahora casi todo estaba en MainActivity.kt. A partir de esta versión repartiremos responsabilidades entre varios archivos.

Archivo	Responsabilidad
MainActivity.kt	Mantendrá el estado compartido y decidirá qué pantalla debe mostrarse.
ConsultaScreen.kt	Contendrá la interfaz de búsqueda que ya teníamos en 0.2.
FavoritosScreen.kt	Mostrará por ahora una pantalla provisional.
ConfiguracionScreen.kt	Mostrará por ahora una pantalla provisional.
build.gradle.kts (Module :app)	Añadirá la biblioteca de navegación.

RECUERDA. En Android Studio cada archivo .kt tiene sus propios imports. Que una clase esté importada en MainActivity.kt no significa que también lo esté en ConsultaScreen.kt.

4. Añadimos la biblioteca de navegación

CONCEPTO NUEVO · Biblioteca
Una biblioteca es código ya preparado que añadimos al proyecto para no tener que programarlo todo desde cero. Navigation Compose aporta las piezas necesarias para movernos entre pantallas Compose.

3	**Abre build.gradle.kts del módulo app.** Busca el bloque dependencies { ... }. No utilices el archivo Gradle de nivel de proyecto.

ARCHIVO: build.gradle.kts (Module :app)	**UBICACIÓN EXACTA:** dentro de dependencies { ... }, junto a las demás líneas implementation(...)

```
implementation("androidx.navigation:navigation-compose:2.10.2")
```

4	**Sincroniza Gradle.** Pulsa Sync Now si Android Studio lo solicita. No continúes hasta que la sincronización termine sin errores.

COMPRUEBA AHORA. No esperamos todavía ningún cambio visual. Solo hemos incorporado al proyecto las herramientas de navegación que utilizaremos después.

5. Separamos la pantalla de Consulta

CONCEPTO NUEVO · Separar responsabilidades

Hasta ahora MeteoLocalApp() guardaba el estado y dibujaba toda la interfaz. A partir de ahora separaremos esas tareas: MeteoLocalApp() conservará los datos compartidos y ConsultaScreen() se ocupará de mostrar la pantalla de consulta.

```
ANTES
MeteoLocalApp()
├─ estado
└─ interfaz

DESPUÉS
MeteoLocalApp()
├─ estado
└─ navegación

ConsultaScreen()
└─ interfaz de consulta
```

5	**Crea ConsultaScreen.kt.** En el paquete principal, crea un archivo Kotlin llamado ConsultaScreen.kt.

ARCHIVO: ConsultaScreen.kt	**UBICACIÓN EXACTA:** después de package e imports

```
@Composable
fun ConsultaScreen(
    localidad: String,
    mensaje: String,
    onLocalidadChange: (String) -> Unit,
    onConsultar: () -> Unit,
    onSolicitarLimpiar: () -> Unit
) {
    // Aquí trasladaremos la interfaz de MeteoLocal 0.2.
}
```

Los parámetros tienen dos funciones diferentes: **localidad y mensaje** traen datos a la pantalla; los parámetros que empiezan por **on...** representan acciones que la pantalla comunica hacia fuera.

CONCEPTO NUEVO · Callback

Un callback es una función que pasamos como parámetro para que otro componente pueda avisarnos de que ha ocurrido algo. Por ejemplo, ConsultaScreen no modificará directamente localidad; avisará mediante onLocalidadChange.

```
DATOS
MeteoLocalApp ──────────────────► ConsultaScreen

EVENTOS
MeteoLocalApp ◄────────────────── ConsultaScreen
```

6	**Mueve la interfaz de 0.2.** Desde MainActivity.kt, mueve el bloque Column(...) que dibujaba la pantalla a ConsultaScreen(), dentro de las llaves de la función. No muevas las declaraciones var localidad ni var mensaje.

ANDROID STUDIO. Si al mover el código aparecen nombres en rojo (Column, Text, Button, Modifier, etc.), utiliza Alt+Enter para importar cada componente en ConsultaScreen.kt. Los imports pertenecen al archivo, no al proyecto completo.

6. Adaptamos los controles de ConsultaScreen

La interfaz que has movido todavía intenta modificar directamente variables que ya no pertenecen a ConsultaScreen. Vamos a conectarla con los callbacks.

7	**Adapta el campo Localidad.** Busca el OutlinedTextField dentro de la Column y sustituye su onValueChange.

ARCHIVO: ConsultaScreen.kt	**UBICACIÓN EXACTA:** dentro de Column(...), en el OutlinedTextField existente

```
OutlinedTextField(
    value = localidad,
    onValueChange = { nuevoValor: String ->
        onLocalidadChange(nuevoValor)
    },
    label = { Text("Localidad") },
    singleLine = true,
    modifier = Modifier.fillMaxWidth()
)
```

¿Qué ocurre aquí? Cuando el usuario escribe, OutlinedTextField recibe el nuevo texto. ConsultaScreen no lo guarda: llama a onLocalidadChange(nuevoValor) para comunicar el cambio a MeteoLocalApp.

8	**Adapta Consultar.** Sustituye el botón Consultar de 0.2 por un botón que llame al callback recibido.

```
Button(onClick = onConsultar) {
    Text("Consultar")
}
```

9	**Adapta Limpiar.** Sustituye el botón Limpiar por una llamada a onSolicitarLimpiar. En esta versión no borraremos inmediatamente: pediremos confirmación.

```
Button(onClick = onSolicitarLimpiar) {
    Text("Limpiar")
}
```

COMPRUEBA AHORA. ConsultaScreen ya sabe mostrar la interfaz y comunicar acciones, pero todavía no puede ejecutarse sola. En unos pasos MeteoLocalApp le proporcionará los datos y las acciones.

7. Creamos Favoritos y Configuración

10	**Crea FavoritosScreen.kt.** Por ahora solo necesitamos comprobar que podemos navegar hasta esa pantalla.

```
@Composable
fun FavoritosScreen() {
    Text("Todavía no tienes localidades favoritas")
}
```

11	**Crea ConfiguracionScreen.kt.** También será provisional en esta versión.

```
@Composable
fun ConfiguracionScreen() {
    Text("Configuración de MeteoLocal")
}
```

CONCEPTO NUEVO · Pantallas provisionales

No estamos perdiendo el tiempo con pantallas vacías: nos permiten comprobar primero la arquitectura de navegación. Más adelante añadiremos persistencia y contenido real sin tener que cambiar la estructura básica.

8. Conservamos el estado en MeteoLocalApp

CONCEPTO NUEVO · State hoisting o elevación del estado

Queremos que localidad y mensaje sigan existiendo aunque salgamos de Consulta y visitemos Favoritos. Por eso el estado se mantiene en un nivel superior, MeteoLocalApp(), y la pantalla recibe los valores como parámetros.

```
MeteoLocalApp
  ├─ localidad
  ├─ mensaje
  └─ muestra una pantalla

      ↓ datos
ConsultaScreen
```

12	**Vuelve a MainActivity.kt.** Dentro de MeteoLocalApp(), conserva localidad y mensaje al comienzo de la función.

ARCHIVO: MainActivity.kt → MeteoLocalApp()	**UBICACIÓN EXACTA:** primeras líneas de la función, inmediatamente después de {

```
var localidad by remember { mutableStateOf("") }

var mensaje by remember {
    mutableStateOf("Introduce una localidad")
}
```

13	**Añade el estado del diálogo Limpiar.** Lo declaramos ahora, antes de utilizarlo.

```
var mostrarDialogoLimpiar by remember {
    mutableStateOf(false)
}
```

Este Boolean indica únicamente si el diálogo debe estar visible: **false** = oculto; **true** = visible.

9. Introducimos NavController

CONCEPTO NUEVO · NavController

NavController es el objeto que controla los cambios de pantalla. Podemos pensar en él como el coordinador de la navegación: cuando le pedimos navigate("favoritos"), cambia el destino actual a Favoritos.

```
Usuario pulsa Favoritos
      ↓
navController.navigate("favoritos")
      ↓
cambia el destino actual
```

14	**Crea NavController.** Añádelo después de los estados, antes de construir la interfaz.

ARCHIVO: MainActivity.kt → MeteoLocalApp()	**UBICACIÓN EXACTA:** después de mostrarDialogoLimpiar

```
val navController = rememberNavController()
```

15	**Obtén la ruta actual.** Necesitamos saber qué destino está activo para marcar correctamente la opción seleccionada en la barra inferior.

```
val backStackEntry by
   navController.currentBackStackEntryAsState()

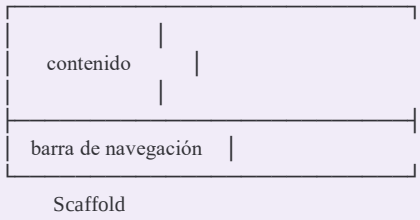
val rutaActual =
   backStackEntry?.destination?.route
```

La palabra **ruta** será simplemente un nombre que identifica una pantalla: "consulta", "favoritos" o "configuracion".

10. Construimos el esqueleto visual con Scaffold

CONCEPTO NUEVO · Scaffold

Scaffold es un componente Material que organiza zonas habituales de una pantalla. Nosotros lo usaremos para reservar una barra de navegación inferior y una zona de contenido principal.



16	**Añade Scaffold.** Colócalo después de rutaActual. La interfaz antigua ya no debe seguir aquí: se movió a ConsultaScreen.

ARCHIVO: MainActivity.kt → MeteoLocalApp()	**UBICACIÓN EXACTA:** después de rutaActual

```
Scaffold(
    bottomBar = {
        NavigationBar {
            NavigationBarItem(
                selected = rutaActual == "consulta",
                onClick = {
                    navController.navigate("consulta")
                },
                icon = { Text("C") },
                label = { Text("Consulta") }
            )

            NavigationBarItem(
                selected = rutaActual == "favoritos",
                onClick = {
                    navController.navigate("favoritos")
                },
                icon = { Text("F") },
                label = { Text("Favoritos") }
            )

            NavigationBarItem(
                selected = rutaActual == "configuracion",
                onClick = {
                    navController.navigate("configuracion")
                },
                icon = { Text("A") },
                label = { Text("Ajustes") }
            )
        }
    }
) { innerPadding ->
    // Aquí colocaremos el contenido en el paso siguiente.
}
```

Dos líneas importantes:

- selected = rutaActual == "consulta" devuelve true cuando Consulta es la pantalla actual y permite resaltarla.
- navController.navigate("favoritos") solicita cambiar al destino cuyo nombre de ruta es "favoritos".

CONCEPTO NUEVO · innerPadding

Scaffold informa al contenido del espacio ocupado por sus barras. innerPadding es ese margen interior. Lo aplicaremos al contenido para evitar que quede escondido detrás de la barra inferior.

11. Relacionamos rutas y pantallas con NavHost

CONCEPTO NUEVO · NavHost

NavController sabe a qué ruta queremos ir, pero necesita un mapa que indique qué pantalla corresponde a cada ruta. Ese mapa es NavHost.

"consulta" → ConsultaScreen()
"favoritos" → FavoritosScreen()
"configuracion" → ConfiguracionScreen()

17	**Inserta NavHost dentro de Scaffold.** Sustituye el comentario que dejaste dentro de { innerPadding -> ... } por este bloque.

ARCHIVO: MainActivity.kt → MeteoLocalApp()	**UBICACIÓN EXACTA:** DENTRO de Scaffold, entre { innerPadding -> y la llave que cierra Scaffold

```
NavHost(
    navController = navController,
    startDestination = "consulta",
    modifier = Modifier.padding(innerPadding)
) {
    composable("consulta") {
        ConsultaScreen(
            localidad = localidad,
            mensaje = mensaje,
            onLocalidadChange = { nuevoValor: String ->
                localidad = nuevoValor
            },
            onConsultar = {
                mensaje =
                    if (localidad.isBlank()) {
                        "Introduce una localidad"
                    } else {
                        "Consulta preparada para: $localidad"
                    }
            },
            onSolicitarLimpiar = {
                mostrarDialogoLimpiar = true
            }
        )
    }

    composable("favoritos") {
        FavoritosScreen()
    }

    composable("configuracion") {
        ConfiguracionScreen()
    }
}
```

Lee el primer destino con calma: composable("consulta") significa «cuando la ruta actual sea consulta, muestra ConsultaScreen». Además le entregamos localidad, mensaje y las acciones que necesita.

COMPRUEBA AHORA. Ejecuta ahora MeteoLocal. Debes poder cambiar entre Consulta, Favoritos y Ajustes. Escribe Sevilla en Consulta, navega a otra pantalla y vuelve: el texto debe conservarse. Si esto funciona, has completado el HITO 1: navegación multipantalla.

12. HITO 2: acciones sin cambiar de pantalla

Hasta ahora hemos trabajado con navegación: cambiar de una pantalla a otra. Ahora añadiremos interacciones que aparecen encima de la pantalla actual sin navegar a otro destino.

```
NAVEGACIÓN
Consulta → Favoritos → Configuración

NO ES NAVEGACIÓN
Consulta + diálogo
Consulta + menú desplegable
```

13. Diálogo de confirmación para Limpiar

CONCEPTO NUEVO · AlertDialog

AlertDialog muestra una ventana modal encima de la interfaz. La usaremos para evitar que Limpiar borre la consulta accidentalmente.

18	**Añade el diálogo en MeteoLocalApp.** Debe ir después de cerrar Scaffold, pero antes de la llave final de MeteoLocalApp(). Así puede acceder a localidad, mensaje y mostrarDialogoLimpiar.

ARCHIVO: MainActivity.kt → MeteoLocalApp()	**UBICACIÓN EXACTA:** después de Scaffold y antes de la última } de la función

```
if (mostrarDialogoLimpiar) {
    AlertDialog(
        onDismissRequest = {
            mostrarDialogoLimpiar = false
        },
        title = {
            Text("Limpiar consulta")
        },
        text = {
            Text("¿Deseas borrar la localidad introducida?")
        },
        confirmButton = {
            TextButton(
                onClick = {
                    localidad = ""
                    mensaje = "Introduce una localidad"
                    mostrarDialogoLimpiar = false
                }
            ) {
                Text("Sí, limpiar")
            }
        },
        dismissButton = {
            TextButton(
                onClick = {
                    mostrarDialogoLimpiar = false
                }
            ) {
                Text("Cancelar")
            }
        }
    )
}
```

El recorrido ahora es:

```
Usuario pulsa Limpiar
        ↓
onSolicitarLimpiar()
        ↓
mostrarDialogoLimpiar = true
        ↓
AlertDialog aparece
    ├─ Cancelar → false
    └─ Confirmar → borra datos y false
```

COMPRUEBA AHORA. Prueba Limpiar. Debe aparecer el diálogo. Cancelar no debe borrar nada; confirmar sí.

14. Menú «Más» en ConsultaScreen

CONCEPTO NUEVO · DropdownMenu
DropdownMenu es un menú desplegable asociado a un elemento de la interfaz. En esta práctica la palabra «Más» será un botón de texto que abre una opción: «Acerca de MeteoLocal». Más adelante podríamos sustituirlo por un icono de tres puntos.

19	**Crea dos estados locales en ConsultaScreen.** Estos estados solo interesan a la pantalla Consulta, por eso no necesitan estar en MeteoLocalApp.

ARCHIVO: ConsultaScreen.kt → ConsultaScreen()	**UBICACIÓN EXACTA:** inmediatamente después de la llave de apertura y antes de Column(...)

```
var menuAbierto by remember {
    mutableStateOf(false)
}

var mostrarAcercaDe by remember {
    mutableStateOf(false)
}
```

IMPORTANTE. Como estas variables utilizan by en ConsultaScreen.kt, comprueba que ESTE ARCHIVO tenga también los imports getValue y setValue. Los imports de MainActivity.kt no se comparten.

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
```

20	**Añade el acceso «Más».** Dentro de la Column de ConsultaScreen, colócalo después del título MeteoLocal y antes del primer Spacer.

ARCHIVO: ConsultaScreen.kt → Column(...)	**UBICACIÓN EXACTA:** justo después del Text que muestra «MeteoLocal»

```
Box {
    TextButton(
        onClick = {
            menuAbierto = true
        }
    ) {
        Text("Más")
    }

    DropdownMenu(
        expanded = menuAbierto,
        onDismissRequest = {
            menuAbierto = false
        }
    ) {
        DropdownMenuItem(
            text = {
                Text("Acerca de MeteoLocal")
            },
            onClick = {
                menuAbierto = false
                mostrarAcercaDe = true
            }
        )
    }
}
```

La palabra «Más» que verás en la app es, por tanto, un **TextButton**, no un subtítulo. Al pulsarlo, menuAbierto pasa a true y DropdownMenu se hace visible.

15. Diálogo «Acerca de»

21	**Añade el diálogo informativo.** Ponlo después de cerrar la Column de ConsultaScreen, pero antes de la llave final de ConsultaScreen().

ARCHIVO: ConsultaScreen.kt → ConsultaScreen()	**UBICACIÓN EXACTA:** después de Column(...) y antes de la última } de la función

```
if (mostrarAcercaDe) {
    AlertDialog(
        onDismissRequest = {
            mostrarAcercaDe = false
        },
        title = {
            Text("MeteoLocal")
        },
        text = {
            Text(
                "Aplicación desarrollada durante " +
                "la Unidad 2 de PMDM."
            )
        },
        confirmButton = {
            TextButton(
                onClick = {
                    mostrarAcercaDe = false
                }
            ) {
                Text("Cerrar")
            }
        }
    )
}
```

COMPRUEBA AHORA. Pulsa Más → Acerca de MeteoLocal. Debe aparecer el diálogo. Al cerrarlo debes permanecer en Consulta: no se ha producido navegación.

16. Mapa estructural final — para comprender, no para copiar

Este esquema no es código. Úsalo para comprobar que entiendes quién hace cada cosa.

```
MainActivity.kt
└─ MeteoLocalApp()
   ├─ estado compartido
   │  ├─ localidad
   │  ├─ mensaje
   │  └─ mostrarDialogoLimpiar
   ├─ NavController
   ├─ Scaffold
   │  ├─ barra inferior
   │  └─ NavHost
   │     ├─ "consulta"      → ConsultaScreen
   │     ├─ "favoritos"     → FavoritosScreen
   │     └─ "configuracion" → ConfiguracionScreen
   └─ AlertDialog de Limpiar

ConsultaScreen.kt
└─ ConsultaScreen()
   ├─ estados locales del menú
   ├─ interfaz de consulta
   ├─ DropdownMenu
   └─ AlertDialog Acerca de
```

17. Vocabulario mínimo de esta práctica

Término	Idea que debes recordar
@Composable	Función que puede construir una parte de la interfaz.
State hoisting	Mantener el estado en un nivel superior y pasarlo a las pantallas.
Callback	Función que una pantalla usa para comunicar un evento hacia fuera.
NavController	Coordina los cambios de destino.
Ruta	Nombre que identifica un destino, como «consulta».
NavHost	Relaciona cada ruta con la pantalla que debe mostrarse.
Scaffold	Organiza zonas principales de una interfaz Material.
DropdownMenu	Menú desplegable que aparece sin cambiar de pantalla.
AlertDialog	Ventana modal de confirmación o información.

18. Batería mínima de pruebas

Acción	Resultado esperado	Comprobado
Abrir la aplicación	Se muestra Consulta.	☐
Escribir Sevilla	El campo muestra Sevilla.	☐
Pulsar Consultar	Aparece el mensaje correspondiente.	☐
Ir a Favoritos	Se muestra la pantalla provisional.	☐
Ir a Ajustes	Se muestra Configuración.	☐
Volver a Consulta	Se conservan localidad y mensaje.	☐
Pulsar Limpiar	Aparece el diálogo de confirmación.	☐
Cancelar	No se borra la consulta.	☐
Confirmar Limpiar	Se vacían localidad y mensaje.	☐
Más → Acerca de	Aparece el diálogo informativo.	☐
Cerrar Acerca de	Seguimos en Consulta.	☐

19. Preguntas de comprensión

1. ¿Por qué hemos sacado la interfaz de consulta de MeteoLocalApp() y la hemos llevado a ConsultaScreen()?

_____

2. ¿Por qué localidad y mensaje siguen estando en MeteoLocalApp()?

_____

3. Explica con tus palabras para qué sirve NavController.

_____

4. ¿Qué diferencia existe entre NavController y NavHost?

_____

5. ¿Qué función cumple Scaffold en nuestra aplicación?

_____

6. ¿Qué significa que una ruta sea «consulta»?

7. ¿Qué diferencia existe entre navegar a Favoritos y abrir el menú «Más»?

8. ¿Por qué menuAbierto puede ser un estado local de ConsultaScreen()?

9. ¿Qué ocurre desde que el usuario pulsa Limpiar hasta que aparece el diálogo?

20. Comprobación antes de terminar

Comprueba	Hecho
He partido de una versión 0.2 que funcionaba.	☐
Entiendo por qué hemos creado tres funciones @Composable para las tres pantallas.	☐
Entiendo qué datos se mantienen en MeteoLocalApp y por qué.	☐
Sé explicar qué hace NavController.	☐
Sé explicar qué hace NavHost.	☐
Sé explicar para qué utilizamos Scaffold.	☐
La barra inferior navega entre los tres destinos.	☐
La localidad se conserva al cambiar de pantalla.	☐
Limpiar abre un diálogo antes de borrar.	☐
El menú Más abre un DropdownMenu.	☐
Acerca de MeteoLocal abre un diálogo sin navegar.	☐
ConsultaScreen.kt contiene sus propios imports getValue y setValue.	☐
La aplicación compila y funciona en Movil_U1.	☐
He realizado toda la batería mínima de pruebas.	☐

AL TERMINAR. Conserva MeteoLocal 0.3 como nueva versión estable. En la siguiente práctica añadiremos capacidades propias del dispositivo sobre esta estructura de pantallas y navegación.

EVALUACIÓN. Esta práctica forma parte del proceso de aprendizaje. Su posible utilización posterior como evidencia evaluable se decidirá al cerrar la secuencia completa de actividades de la Unidad 2.